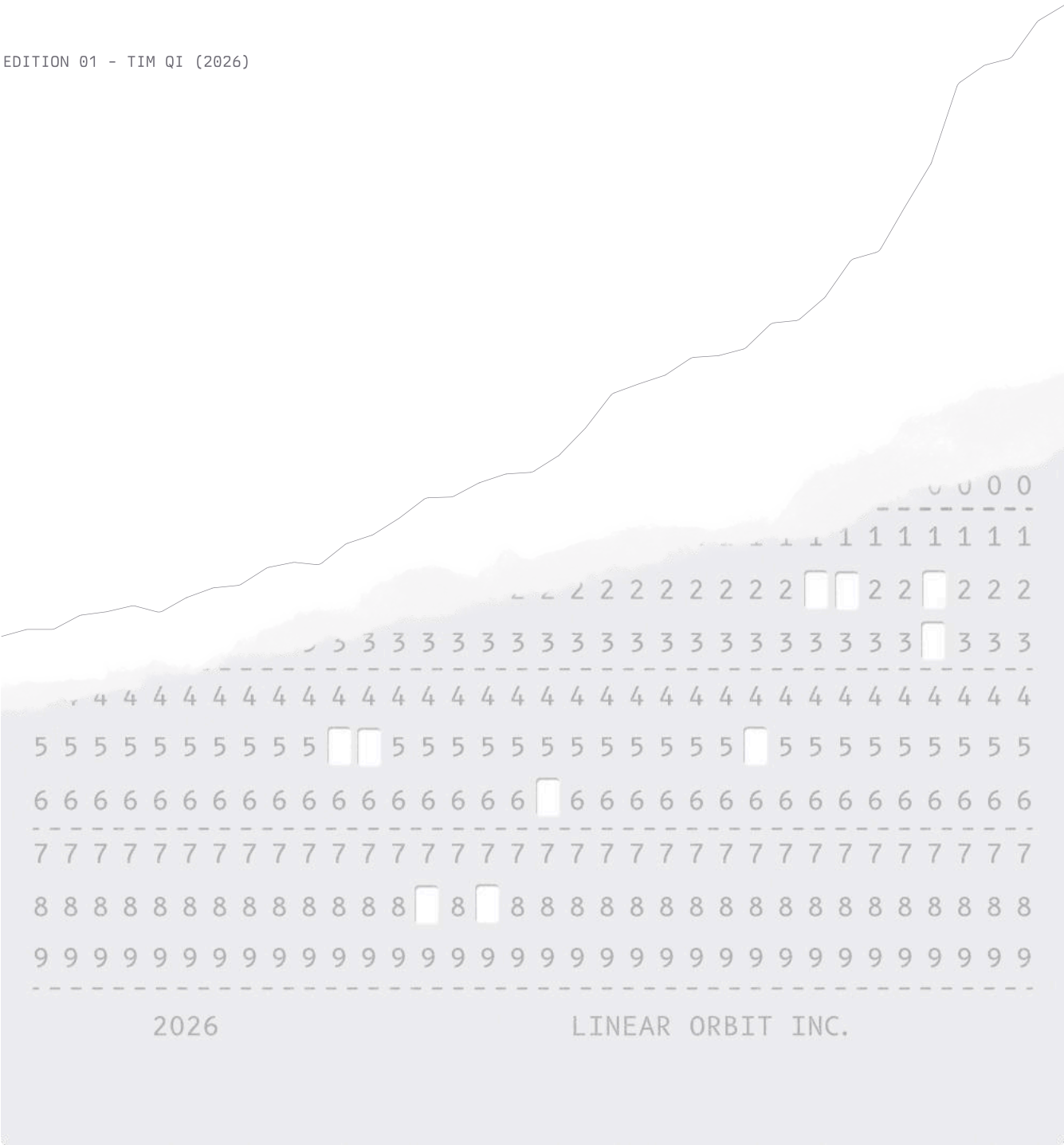




AI usage patterns in software teams

EDITION 01 - TIM QI (2026)





Teams building with coding agents now ship 6.5x as much as those that don't.

Tens of thousands of teams build software inside Linear every day. Over six years that's given us a detailed picture of how product development happens. Lately, AI has become part of that.

We wanted to explore how AI is changing product development by visualizing the shift over time, from before AI was widely adopted to now.

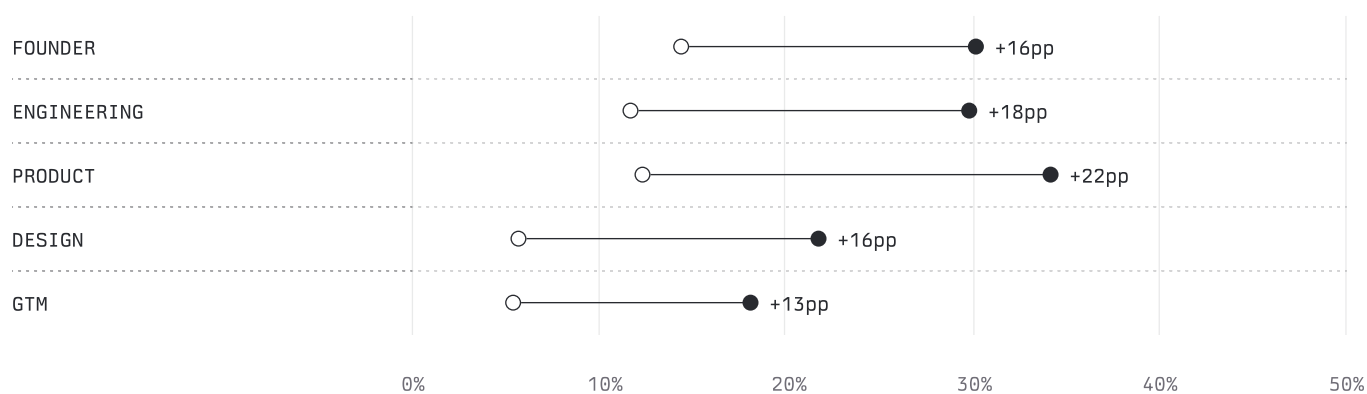
Model companies and coding tools have published plenty on token usage and code volume, but that captures only one layer. We're unusually well placed to see the entire workflow behind building a product.

We look at three things across that transition. Who is using AI, how it reshapes where teams spend their time across Linear, and whether it changes how much they ship.



AI adoption has spread to every function.

Adoption more than doubled across every function. Product grew the fastest, and even go-to-market, traditionally furthest from the codebase, climbed sharply. AI in product development is no longer just an engineering story.

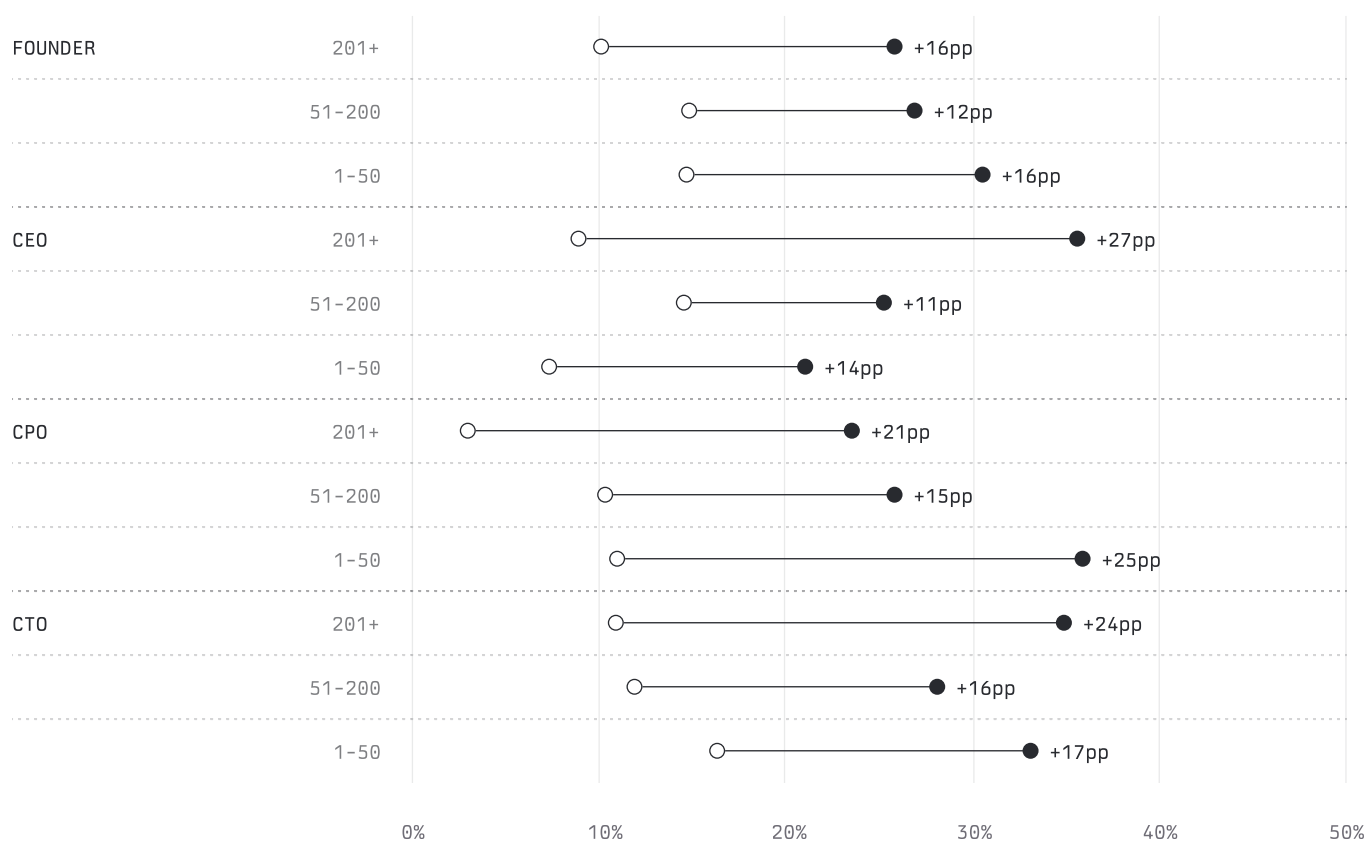


○ JAN 2026 ● JUN 2026 PERCENTAGE OF USERS ACTIVE ON AI FEATURES (LAST 30 DAYS) BY FUNCTION



Adoption goes all the way to the top.

Executives are personally active on AI, often as much as their teams, suggesting that even the most senior leaders are learning the technology by getting hands-on.

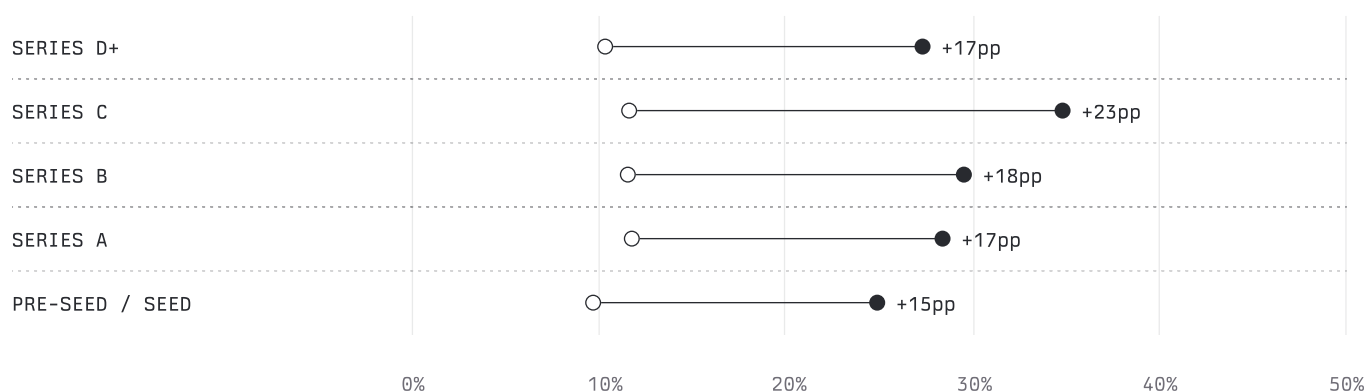


○ JAN 2026 ● JUN 2026 PERCENTAGE OF USERS ACTIVE ON AI FEATURES (LAST 30 DAYS) BY EXECUTIVE TEAM



Adoption is consistent at every stage.

Adoption rose at every funding stage, from pre-seed to Series D+. Company maturity, usually a good predictor of how fast an organization moves on new technology, barely registers here.

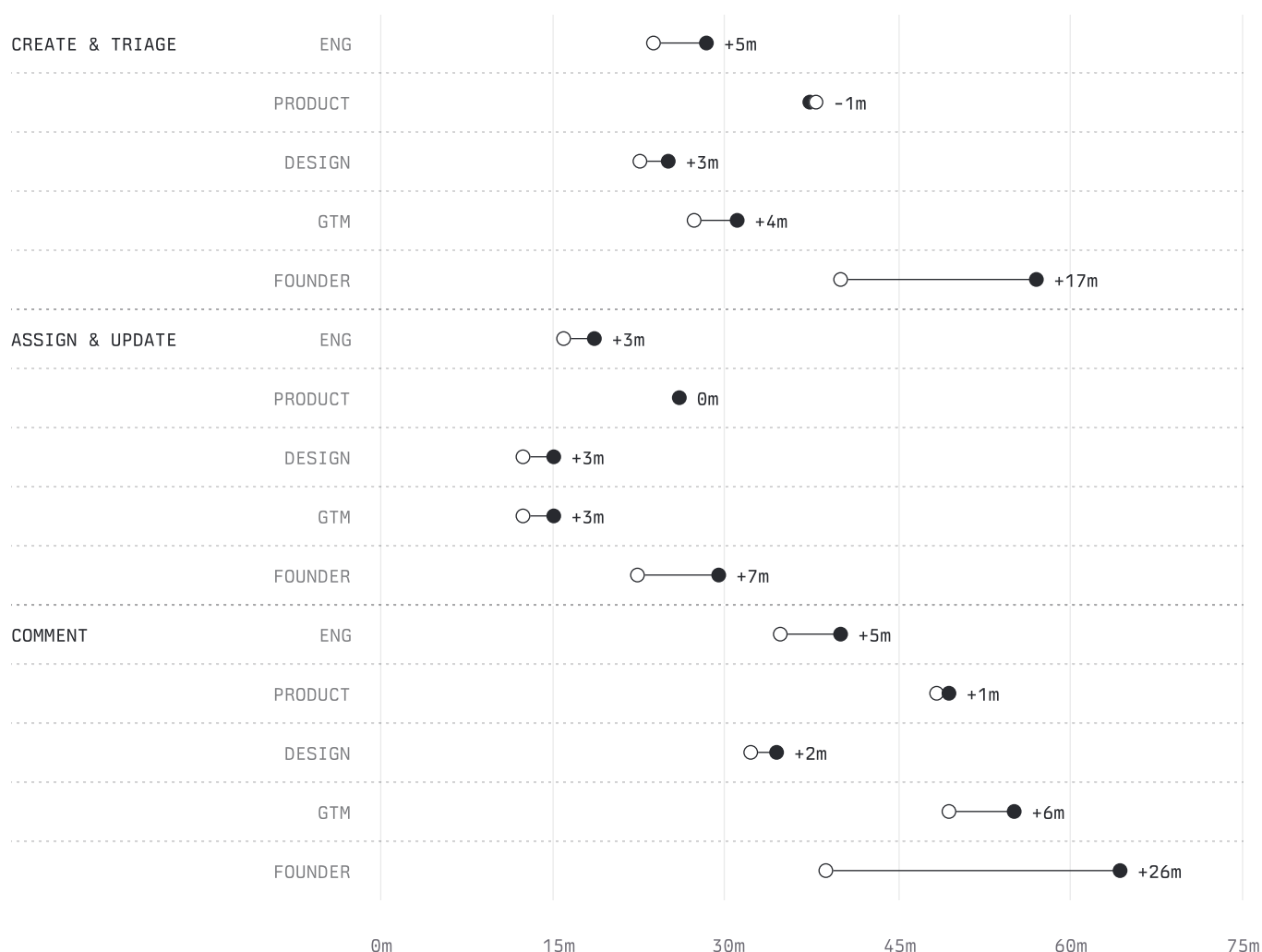


○ JAN 2026 ● JUN 2026 PERCENTAGE OF USERS ACTIVE ON AI FEATURES (LAST 30 DAYS) BY FUNDING STAGE



Teams are putting more into the system.

Time spent creating, triaging, and commenting rose in nearly every function. More work needs more coordination, and coordination now sets the context agents act on.

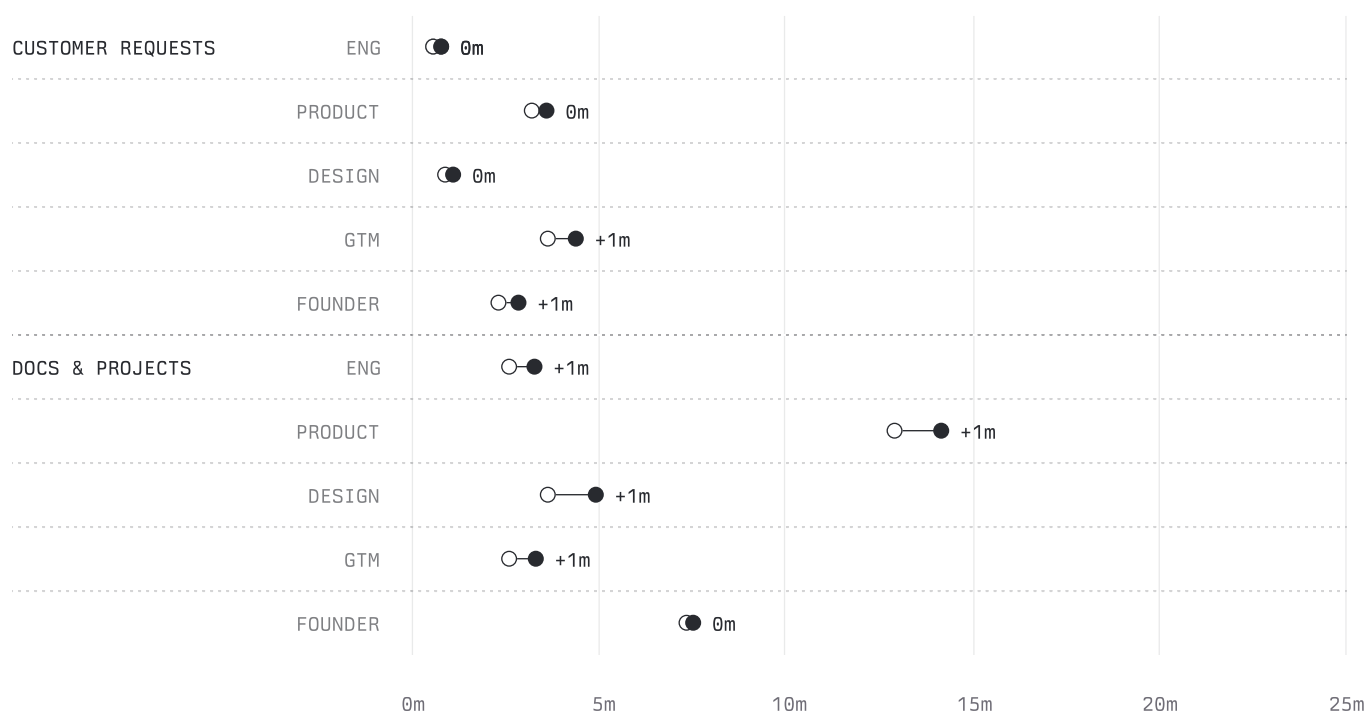


○ JAN 2026 ● JUN 2026 TIME SPENT (MINUTES) BY ACTIVITY AND FUNCTION



Planning time didn't move.

Time spent on customer requests, docs and initiatives barely moved in a year, even though the tools for writing specs and parsing feedback got far better.

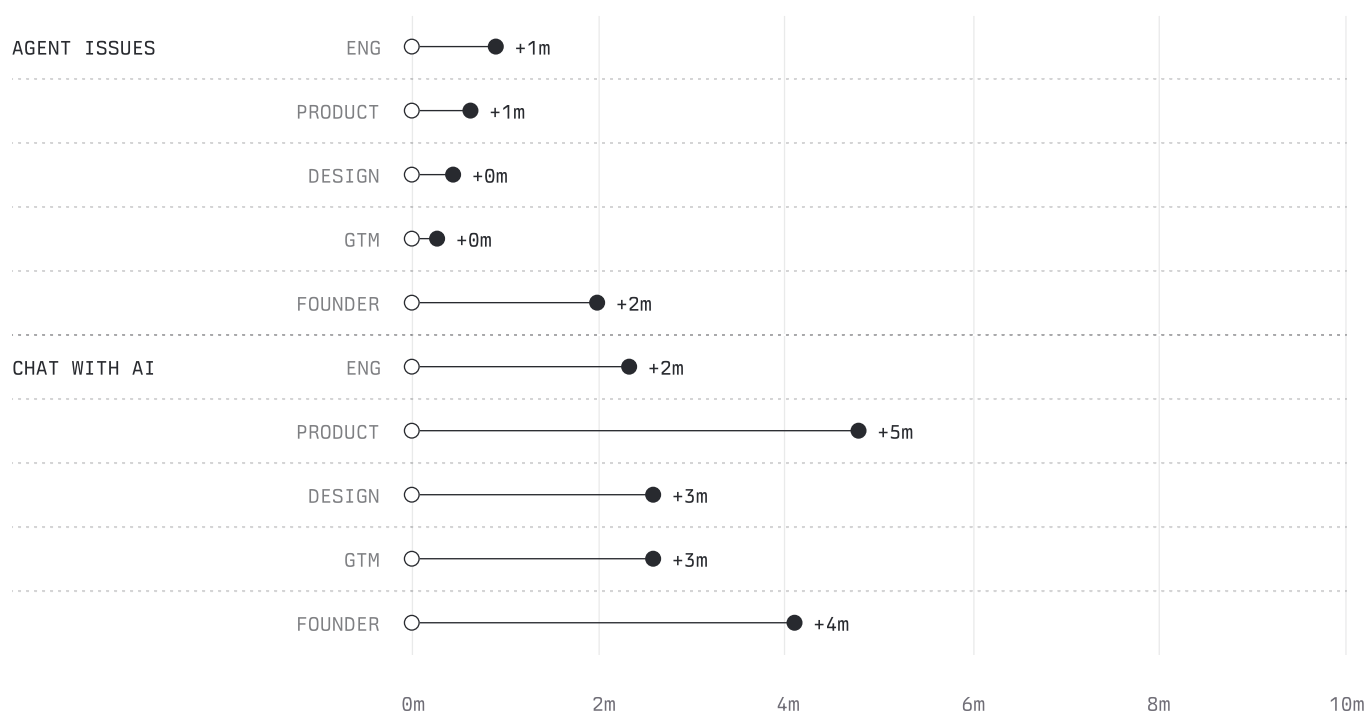


○ JAN 2026 ● JUN 2026 TIME SPENT (MINUTES) BY ACTIVITY AND FUNCTION



A new layer of work appeared.

Chatting with AI and delegating issues to agents are now part of the working week. Nothing else shrank to make room, so this new layer landed on top.

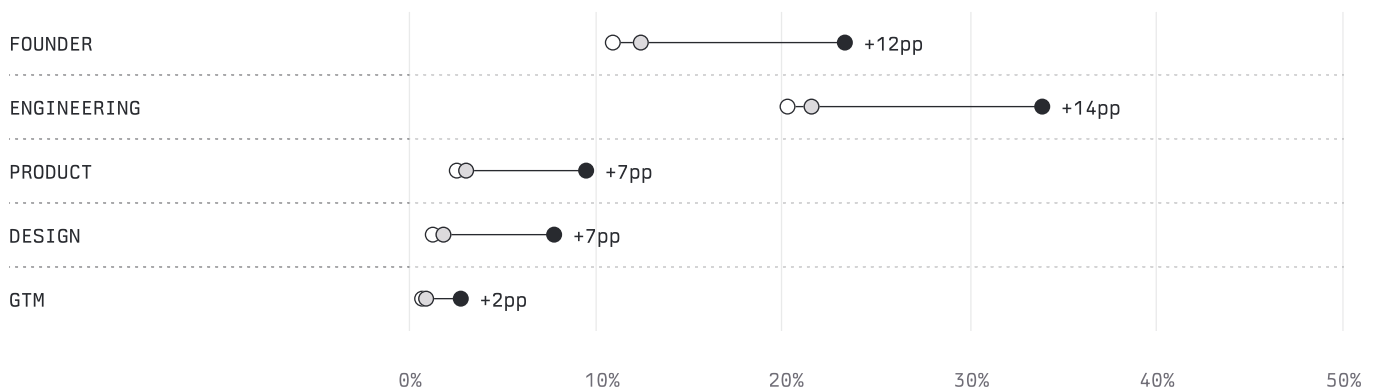


○ JAN 2026 ● JUN 2026 TIME SPENT (MINUTES) BY ACTIVITY AND FUNCTION



Non-engineers are shipping more code.

Product managers and designers attach pull requests at roughly three times the rate of two years ago. The people who used to describe a change now ship it themselves.

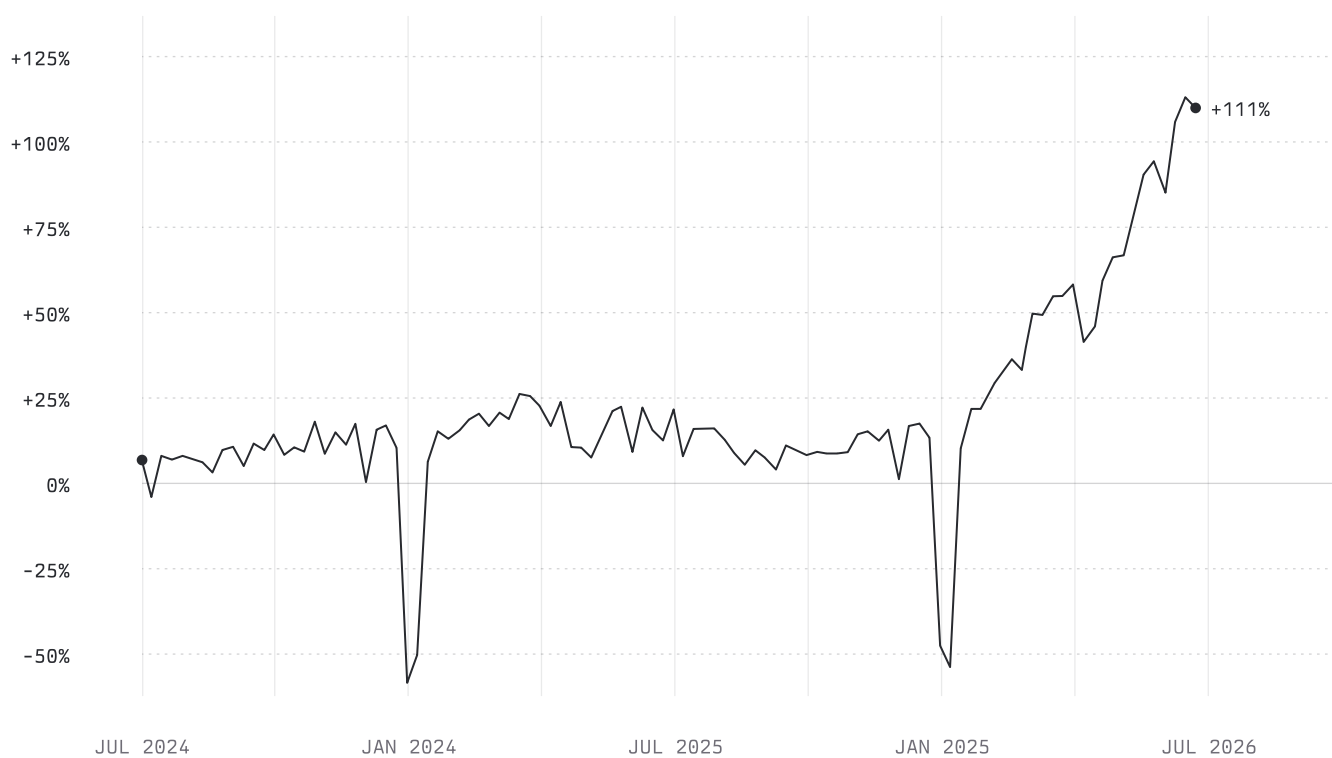


○ JUN 2024 ○ JUN 2025 ● JUN 2026 PERCENTAGE OF USERS WHO ATTACHED A PULL REQUEST (LAST 30 DAYS)



Pull requests are up 111% in two years.

Output held steady for more than a year, then bent upward as model quality and wider adoption climbed together.

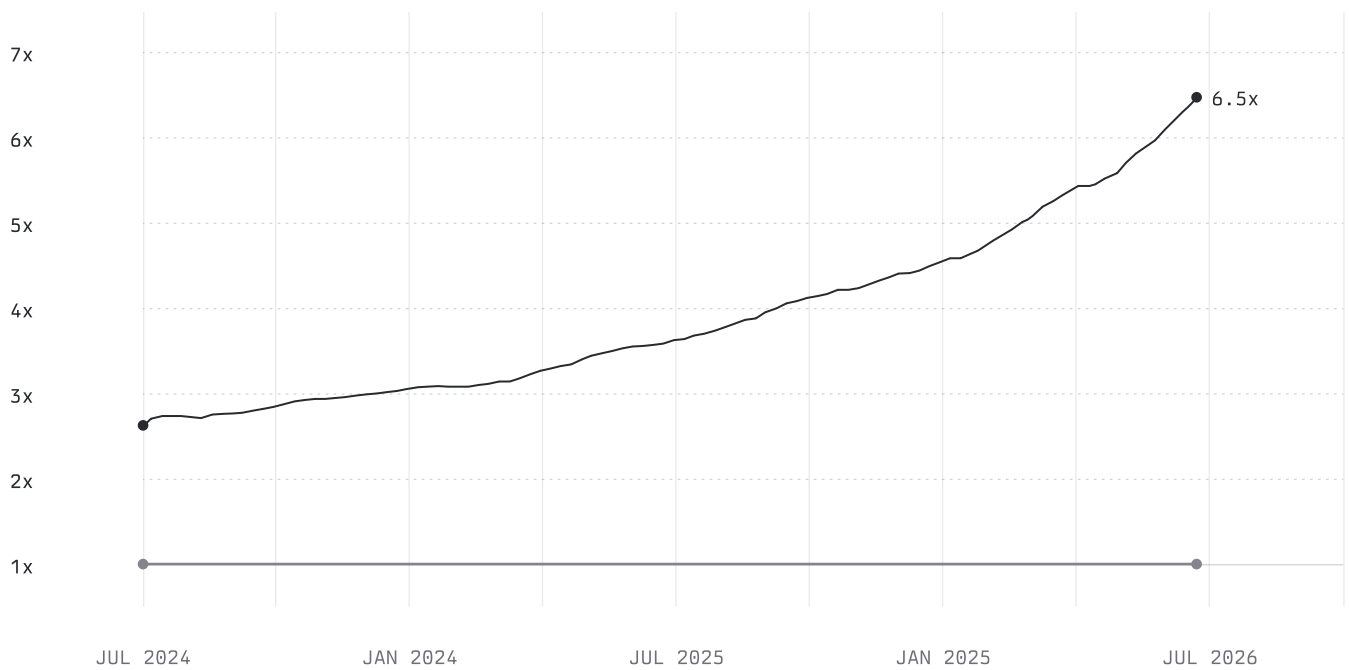


PERCENTAGE CHANGE IN PULL REQUESTS PER TEAM PER WEEK SINCE JUNE 2024 - ALL PAID WORKSPACES



Coding agents account for most of the acceleration.

Teams using coding agents now ship six times as many pull requests as teams without. They were already ahead two years ago; the gap has since more than doubled.

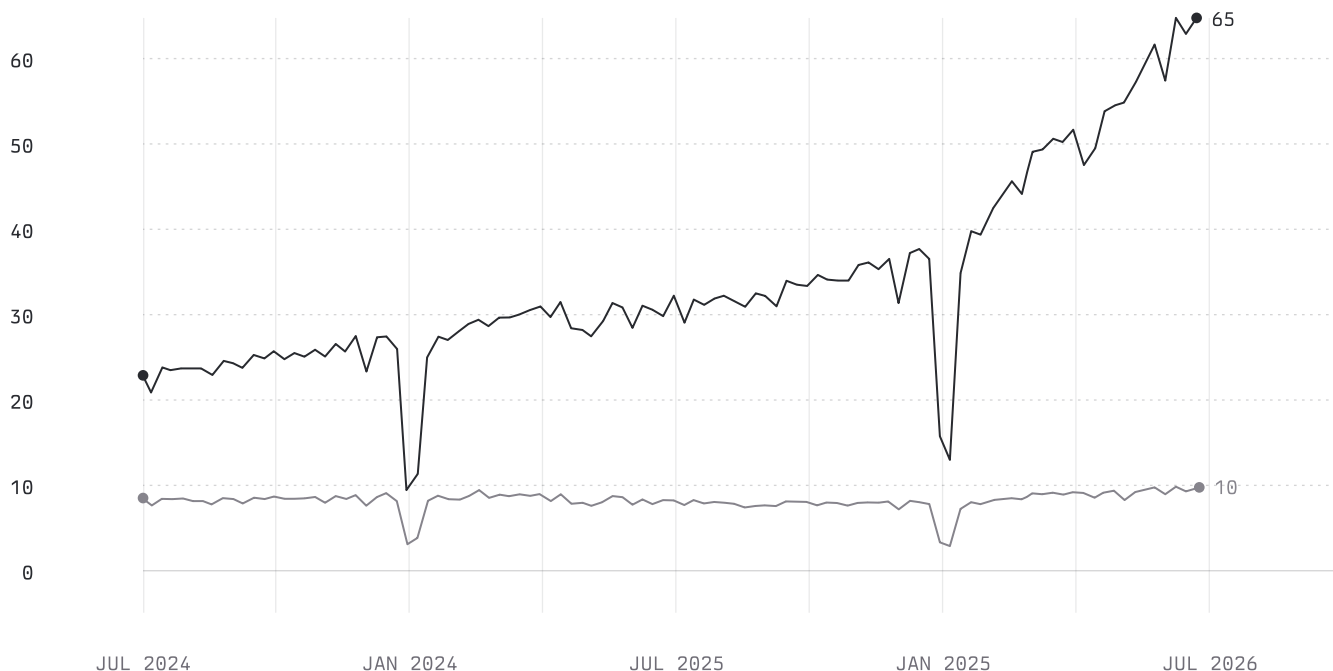


● TRADITIONAL TEAMS ● CODING-AGENT TEAMS RATIO OF AVG PRS PER TEAM PER WEEK



Coding agents account for most of the acceleration.

Teams using coding agents now ship six times as many pull requests as teams without. They were already ahead two years ago; the gap has since more than doubled.



● TRADITIONAL TEAMS ● CODING-AGENT TEAMS PULL REQUESTS PER TEAM PER WEEK - FIXED COHORT (PAID WORKSPACES)



Token spend as a proxy for value will become a relic of AI's early days.

Token spend has been the industry's favorite yardstick, but the conversation is already moving past it, for good reason. A mechanical refactor might burn lots of tokens, while a meaningful bug fix or code review doesn't. Spend and value simply don't line up, and using one as a proxy for the other will be remembered as a relic of AI's early days.

Looking at pull requests offers a clearer signal, because they reflect code that's making it to production, but they still measure motion more than value. To understand how AI is truly influencing product development, we have to observe the process as a whole.

We've recently brought coding capabilities into Linear, alongside Diffs, our native way of reviewing code. Now that the whole arc from customer feedback to deployed code runs through Linear, we're unusually well-placed to see how AI influences product development.

Our bet is that the largest gains from here will come from the application layer, even as models improve. It's the layer that understands the work well enough to aim raw capability at the right tasks, matching each task to the right model without teams having to think about it, because a routine fix doesn't need the most expensive intelligence available. That's how hit-or-miss spend becomes reliable value, and tracing that journey, from token spend through the workflow to the output on the other side, is exactly what we intend to do in future reports.

TIM QI — HEAD OF DATA



Methodology

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since 1966, when designers at Letraset and James Mosley, the librarian at St Bride Printing Library in London, took a 1914 Cicero translation and scrambled it to make dummy text for Letraset's Body Type sheets.

Definitions

AI-active. A user with at least one AI interaction, an in-app or Slack conversation or an agent session, in a 28-day window.

Agent team. A workspace with a coding agent connected.

Pull request. A code change opened against a repository connected to Linear. We count pull requests opened, not merged.

Paid workspace. A workspace on a paid plan, active during the relevant period.

Agent issue. This includes delegating an issue to an agent, starting an agent session

Limitations

Who's in the data. This is Linear's customer base, not software teams everywhere. Teams that choose Linear skew modern and AI-forward, so the patterns here likely run ahead of the wider industry and show up sooner than they do elsewhere.

A low bar for AI adoption. We count someone as AI-active after a single AI session in a month, a deliberately low threshold. We also built the features behind much of this adoption, so its growth is no surprise. The point we're making is about how uniform adoption is across roles, seniority and company size, not the sole fact that it rose.

The agent comparison isn't random. Teams that adopt coding agents tend to be faster and more AI-forward to begin with, so part of the gap reflects who adopts, not the agents. What's harder to explain away is the widening over time, which tracks closely with model improvements and the spread of tools like Claude Code.

Activity, not quality. We can see that code merged, not whether it was good, whether the right things got built, or whether the codebase got healthier. Volume is not the same as value.

Observational, not experimental. None of this is a controlled experiment. The shifts we see often coincide with several things at once, our own feature launches and the broader wave of model progress among them, so we report correlations, not causes.

Some cuts are inferred. Company size and funding stage come from third-party matching that doesn't cover every workspace, and role breakdowns rely on normalising job titles. Both carry some classification error.